This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

HUOPM: High-Utility Occupancy Pattern Mining

Wensheng Gan, Jerry Chun-Wei Lin, *Member, IEEE*, Philippe Fournier-Viger, Han-Chieh Chao, *Senior Member, IEEE*, and Philip S. Yu, *Fellow, IEEE*

***Abstract*—Mining useful patterns from varied types of databases is an important research topic, which has many real-life applications. Most studies have considered the frequency as sole interestingness measure to identify high-quality patterns. However, each object is different in nature. The relative importance of objects is not equal, in terms of criteria, such as the utility, risk, or interest. Besides, another limitation of frequent patterns is that they generally have a low occupancy, that is, they often represent small sets of items in transactions containing many items and, thus, may not be truly representative of these transactions. To extract high-quality patterns in real-life applications, this paper extends the occupancy measure to also assess the utility of patterns in transaction databases. We propose an efficient algorithm named high-utility occupancy pattern mining (HUOPM). It considers user preferences in terms of frequency, utility, and occupancy. A novel frequency-utility tree and two compact data structures, called the utility-occupancy list and frequency-utility table, are designed to provide global and partial downward closure properties for pruning the search space. The proposed method can efficiently discover the complete set of high-quality patterns without candidate generation. Extensive experiments have been conducted on several datasets to evaluate the effectiveness and efficiency of the proposed algorithm. Results show that the derived patterns are intelligible, reasonable, and acceptable, and that HUOPM with its pruning strategies outperforms the state-of-the-art algorithm, in terms of runtime and search space, respectively.**

***Index Terms*—Interesting pattern, utility mining, utility occupancy, utility theory.**

I. Introduction

FREQUENT pattern mining (FPM) and association rule mining (ARM) [1]–[3] are some of the important and fundamental data mining techniques to extract meaningful and useful information from massive amounts of data [4], [5]. FPM is the process of discovering frequent sets of items in transaction databases based on a user-specified minimum *support* threshold. In the recent decades, the task of FPM has been extensively studied by mainly considering the frequency measure for selecting patterns. Other properties and interestingness measures of frequent patterns (FPs) and association rules have also been studied [3] to fulfill the need of discovering more interesting patterns in databases, for example, maximal FPs [6], all-confidence [7], coherence [7], and other constraints [8].

In real-life applications, the importance of objects or patterns is often evaluated in terms of implicit factors, such as the utility, interestingness, weight, and risk or profit [9], [10]. Hence, the knowledge that actually matters to the user may not be found using traditional FPM and ARM algorithms. To measure the utility of patterns, a utility-based mining framework called high-utility pattern mining (HUPM) [10] was proposed, which considers the relative importance of items (*item utility*). It has become an emerging research topic in recent years [11]–[15]. Chan *et al.* [16] first introduced the problem of utility-based pattern mining based on business objectives. Yao *et al.* [17] then defined utility mining as the problem of finding profitable itemsets while considering both the purchase quantities of objects/items in transactions (*internal utilities*) and their unit profits (*external utilities*). In addition, the utility (i.e., importance, interest, or risk) of each object/item can be predefined based on the users' background knowledge or preferences.

Recently, a study [18] has shown that considering the *occupancy* of patterns is critical for many applications. The occupancy measure is used to ensure that each pattern found represents a large part of transactions where it appears. This allows to find patterns that are more representative and, thus, of higher quality. In many applications, occupancy is an interestingness factor to measure a pattern's completeness; and it is an indispensable complement to frequency (or support). However, implicit factors, such as the utility, interestingness, risk or profit of the objects, or patterns are ignored in *occupancy pattern mining* [18], and it ignores the fact that items/objects may appear more than once in transactions. On the other hand, HUPM does not assess the occupancy of patterns. As a result, the discovered patterns may be irrelevant or even misleading if they are not the representative of the supporting transactions. For example, if a pattern occurs in many transactions but its actual occupancy is low, it may be inappropriate to use this pattern for recommendation. Hence, it is desirable to find patterns that are representative of the transactions where they occur. In particular, extracting patterns that occupy a large portion of the utility in their supporting transactions is critical

Manuscript received January 18, 2017; revised August 19, 2018, October 29, 2018 and December 27, 2018; accepted January 25, 2019. This work was supported in part by the Shenzhen Technical Project under Grant JCYJ20170307151733005 and Grant KQJSCX20170726103424709. This paper was recommended by Associate Editor S. Ozawa. *(Corresponding author: Jerry Chun-Wei Lin.)*

W. Gan is with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China, and also with the Department of Computer Science, University of Illinois at Chicago, Chicago, IL 60607 USA (e-mail: wsgan001@gmail.com).

J. C.-W. Lin is with the Department of Computing, Mathematics, and Physics, Western Norway University of Applied Sciences, 5063 Bergen, Norway (e-mail: jerrylin@ieee.org).

P. Fournier-Viger is with the School of Natural Sciences and Humanities, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China (e-mail: philfv@hitsz.edu.cn).

H.-C. Chao is with the Department of Electrical Engineering, National Dong Hwa University, Hualien 97401, Taiwan (e-mail: hcc@ndhu.edu.tw).

P. S. Yu is with the Department of Computer Science, University of Illinois at Chicago, Chicago, IL 60607 USA (e-mail: psyu@uic.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCYB.2019.2896267

2168-2267 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

to several applications. Recently, an algorithm called OCEAN was proposed to address the problem of high-utility occupancy pattern mining (HUOPM) by introducing the *utility occupancy* measure [19]. However, it fails to discover the complete set of high-utility occupancy patterns (HUOPs) and also encounters several performance problems.

Therefore, this paper proposes an effective and efficient algorithm named HUOPM. The proposed algorithm extracts patterns based on users' interests, pattern frequency, and utility occupancy and considers that each item may have a distinct utility. The concept of *utility occupancy* is adopted to evaluate the utility contribution of patterns in their supporting transactions. Thus, the concept of HUOP is quite different from the previous concept of high-utility pattern (HUP). The previous study [18] shows that the *occupancy* of patterns is very critical to some applications. Compared with *occupancy*, *utility occupancy* is suitable and more effective for pattern analysis in some real-life domains, for example, market basket analysis [1], [2], [14]; print-area recommendation for Web pages [18]; Web click analysis [11]; mobile service provider [19]; and biomedical applications [20].

Consider the travel route recommendation for tourist to visit, eat, and spend time/money, HUOPM can successfully exploit the frequency and utility contribution ratio (with respect to utility occupancy) of a specific travel route. For another example, a mobile service provider can obtain some real-world data, and each record contains the traffic information that a customer spends on various mobile Apps. In general, each App has its unit utility (i.e., price and popular). In this case, the concept of the *utility occupancy* and the HUOPM model can be applied to discover the high qualified patterns (both frequent and high-utility occupancy). After identifying the set of mobile Apps that the users are interested in (e.g., download frequently and spend most of their money), this information can also be utilized to improve the service. The major contributions of this paper are summarized as follows.

1) A novel and effective HUOPM algorithm is proposed to address the novel research problem of mining HUOPs with the utility occupancy measure. To the best of our knowledge, no prior algorithms address this problem successfully and effectively.
2) Two compact data structures called utility-occupancy list (UO-list) and frequency-utility table (FU-table) are designed to store the required information about a database, for mining HUOP. When necessary and sufficient conditions are met, information about a pattern can be directly obtained from the built UO-lists of its prefix patterns and, thus, HUOPM can avoid scanning the database repeatedly.
3) Moreover, the concept of remaining utility occupancy is utilized to calculate an upper bound to reduce the search space. Based on the developed pruning strategies, the HUOPM algorithm can directly discover HUOPs from the designed frequency-utility tree (FU-tree) using UO-lists by only scanning a database twice.
4) Extensive experiments have been conducted on both real-world and synthetic datasets to evaluate how effective and efficient the proposed HUOPM algorithm is. Results show that HUOPM can reveal the desired useful patterns, having a high-utility occupancy and predefined frequency, and that the proposed pruning strategies are effective at leading to a more compact search space.

The rest of this paper is organized as follows. Related work is reviewed in Section II. In Section III, some key preliminaries are introduced and the addressed problem is defined. The proposed HUOPM algorithm and several pruning strategies are described in Section IV. Then, to evaluate the effectiveness and efficiency of the HUOPM algorithm, results of extensive experiments comparing its performance with the state-of-the-art algorithm are provided in Section V. Finally, the conclusion and future work are drawn in Section VI.

II. Related Work

A. Support-Based Pattern Mining

In numerous domains, Web mining and data mining technologies provide powerful ways of discovering interesting and useful information in massive amounts of data. In the recent decades, the task of support-based FPM has been widely studied. Most algorithms have been designed to extract patterns using the frequency (support) measure as selection criterion [1]–[3]. Among them, *Apriori* [1] and FP-growth [2] are the most well-known FPM algorithms. Other properties and interestingness measures of FPs and association rules have also been studied to fulfill the need for discovering more interesting patterns [3], [7], [8].

In the past, significant progress have been made concerning the data mining with support and confidence [3]. Many algorithms have been proposed to discover the high-quality patterns from the binary transaction databases [3], [7], [8], [21]. However, these approaches do not consider that objects/items may occur more than once in a transaction (e.g., may have nonbinary purchase quantities), and that objects/items are often not equally important to the user (e.g., may not have the same unit profit). Until now, the associations of Boolean attributes are only considered in traditional ARM, which only reflect the frequency of the presence or absence of an item in the database. Thus, the problem of quantitative ARM (QARM) has been studied [22]–[24]. In QARM, the attributes of item can be quantitative (e.g., age, income, and purchase quantity) instead of the Boolean value (0 or 1). However, QARM still does not reflect the other important factors of items, such as interest, price, risk and profit. In some real-life applications, the frequency (support) of a pattern may be an inappropriate measure to determine the importance of this pattern.

B. Utility-Based Pattern Mining

To address the limitations of support-based pattern mining and to extract high profitable patterns, the HUPM task has been studied [11]–[14]. HUPM considers both the occur quantities and unit profits of objects/items rather than just considering their occurrence frequencies. A pattern is concerned as an HUP if its utility is no less than the predefined minimum utility threshold. Chan *et al.* [16] presented a framework to mine the top-k closed utility patterns. Yao *et al.* [17] then defined utility mining as the problem of discovering profitable patterns while considering both the purchase quantity of objects/items in transactions (*internal utility*) and their unit

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

profits (*external utility*). Then, they introduced the mathematical properties of utility constraint to, respectively, reduce the search space and expected utility upper bounds [25]. The concept of HUPM is different from the support-based weighted FPM (WFPM) [9], [26], [27], because WFPM does not consider the quantity information of each object/item and the range of weight is [0, 1]. By taking both the quantities and profits of items into consideration, HUPM can reveal more valuable patterns than frequent ones. Liu *et al.* [28] then presented a transaction-weighted utilization (TWU) model to discover HUPs by adopting a transaction-weighted downward closure property. In recent years, developing algorithms for mining HUPs is an active research topic. Some recent algorithms for HUPM are IHUP [11], UP-growth [13], UP-growth+ [14], HUI-Miner [12], d2HUP [29], FHM [30], HUP-Miner [31], and EFIM [32].

Different from the efficiency issue of HUPM, there are also a number of studies that focus on some interesting effectiveness issues of HUPM. For example, mining high-utility patterns from different types of data (i.e., uncertain data [33], [34]; temporal data [35], [36]; transaction data with negative unit profits [37]; dynamic data [38]–[40]; and stream data [41]), HUPM with various discount strategies [42], [43], nonredundant correlated HUPs [44], HUPM using multiple minimum utility thresholds [33], a condensed set of HUPs [45], discriminative HUPs [46], top-k issue of HUPM [45], and HUPM from big data [47]. All of these HUPM algorithms discover HUPs based on the basic definitions of utility mining model by Yao *et al.* [17], [25]. In recent years, there has been a growing interest in utility-oriented mining models for discovering different types of HUPs (e.g., itemsets, rules, sequences, and episodes) and profitable information. The comprehensive survey of this research field can be referred to [10] and [40].

C. High-Quality Pattern Mining

Although HUPM methods can evaluate the utility of patterns, they do not assess the occupancy of patterns in their supporting transactions in terms of utility. In other words, HUPM methods do not consider how important the utility of patterns are compared to the utility of the transactions where they appear. For some real-life applications, such as pattern-based recommendation, it is desirable that interesting patterns should occupy a large portion of the transactions where they appear [18]. The above approaches fail to meet this requirement. To address this issue, a novel measure called *utility occupancy* was proposed in the OCEAN algorithm [19]. As a relative measure, utility occupancy ensures that a certain set of items is important to individual users.

However, the OCEAN algorithm suffers from two important drawbacks. First, the mining results derived by OCEAN are incomplete. The reason is that the exact utility information is incorrectly kept in the utility-list [12] structure using an inconsistent sorting order. As a result, OCEAN applies pruning strategies with incorrect information. Second, OCEAN is not efficient as it does not utilize the support property and utility occupancy property well to prune the search space. As a result, it performs poorly when the related parameters are set low. Hence, developing an effective and efficient algorithm to address these limitations is important. In this paper, the problem of effectively mining the complete set of all HUOPs from a database is discussed.

TABLE I
EXAMPLE QUANTITATIVE DATABASE

tid	Transaction (item, quantity)	*tu*
T_1	a:2, c:4, d:7	\$65
T_2	b:2, c:3	\$37
T_3	a:3, b:2, c:1, d:2	\$38
T_4	b:4, d:3	\$11
T_5	a:1, b:3, c:2, d:5, e:1	\$49
T_6	c:2, e:4	\$58
T_7	c:2, d:1	\$23
T_8	a:3, b:1, d:2, e:4	\$61
T_9	a:2, c:4, d:1	\$59
T_{10}	c:3, e:1	\$42

III. PRELIMINARY AND PROBLEM STATEMENT

Let $I = \{i_1, i_2, \ldots, i_m\}$ be a finite set of m distinct items in a transactional quantitative database $D = \{T_1, T_2, \ldots, T_n\}$, where each quantitative transaction $T_q \in D$ is a subset of I, and has a unique identifier *tid*. The total utility of all the items in a transaction is named as transaction utility *tu*. An itemset X with k distinct items $\{i_1, i_2, \ldots, i_k\}$ is called a k-itemset. A database consisting of ten transactions and five distinct items is shown in Table I, which will be used as a running example.

Definition 1: The number of transactions containing an itemset is said to be its occurrence frequency or *support count* [1], [2]. The support count of an itemset X, denoted as $sup(X)$, is the number of *supporting transactions* containing X. Let the set of transactions supporting an itemset X denote as Γ_X. A transaction T_q is said to support an itemset X if $X \subseteq T_q$. Thus, $sup(X) = |\Gamma_X|$. Let the user-specified minimum support threshold denote as α, X is called an frequent pattern (FP) in a database D if $sup(X) \geq \alpha \times |D|$.

Definition 2: Each item i_m in a database D has a unit profit, denoted as $pr(i_m)$, which represents its relative importance to the user. Item unit profits are indicated in a user-specified profit-table, denoted as $ptable = \{pr(i_1), pr(i_2), \ldots, pr(i_m)\}$. The utility of an item i_j in a transaction T_q is defined as $u(i_j, T_q) = q(i_j, T_q) \times pr(i_j)$, in which $q(i_j, T_q)$ is the occur quantity of i_j in T_q. The utility of an itemset/pattern X in a transaction T_q is defined as $u(X, T_q) = \sum_{i_j \in X \wedge X \subseteq T_q} u(i_j, T_q)$. Thus, the total utility of X in a database D is $u(X) = \sum_{X \subseteq T_q \wedge T_q \in D} u(X, T_q)$.

Example 1: In Table I, assume that the unit profit of items (a) to (e) are defined as $\{pr(a)$:\$7, $pr(b)$:\$2, $pr(c)$:\$11, $pr(d)$:\$1, $pr(e)$:\$9$\}$, respectively. Consider the itemsets (a) and (ab), their utilities in T_3 are $u(a, T_3) = 3 \times \$7 = \$21$ and $u(ab, T_3) = 3 \times \$7 + 2 \times \$2 = \$21 + \$4 = \$25$, respectively. Thus, their utilities in the database are calculated as $u(a) = u(a, T_1) + u(a, T_3) + u(a, T_5) + u(a, T_8) + u(a, T_9) = \$14 + \$21 + \$7 + \$21 + \$14 = \$77$, and $u(ab) = u(ab, T_3) + u(ab, T_5) + u(ab, T_8) = \$21 + \$13 + \$23 = \$57$.

Definition 3: The transaction utility (*tu*) of a transaction T_q is $tu(T_q) = \sum_{i_j \in T_q} u(i_j, T_q)$, where i_j is the j-th item in T_q.

Example 2: $tu(T_1) = u(a, T_1) + u(c, T_1) + u(d, T_1) = \$14 + \$44 + \$7 = \$65$. The transaction utilities of transactions

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

T_1 to T_{10} are, respectively, calculated as $tu(T_1) = \$65$, $tu(T_2) = \$37$, $tu(T_3) = \$38$, $tu(T_4) = \$11$, $tu(T_5) = \$49$, $tu(T_6) = \$58$, $tu(T_7) = \$23$, $tu(T_8) = \$61$, $tu(T_9) = \$59$, and $tu(T_{10}) = \$42$, as shown in Table I.

In the study [18], a new interestingness measure called *occupancy* was proposed to discover the FPs having a strong occupancy. A new concept named *utility occupancy* [19] is first introduced below. It is important to notice that the *utility occupancy* concept is different from the original *occupancy* concept presented in the DOFIA algorithm [18].

Definition 4: The utility occupancy of an itemset X in a supporting transaction T_q is denoted as $uo(X, T_q)$, and defined as the ratio of the utility of X in that transaction divided by the total utility of that transaction

$$uo(X, T_q) = \frac{u(X, T_q)}{tu(T_q)}. \tag{1}$$

Example 3: Since $tu(T_1) = \$65$ and $tu(T_3) = \$38$, the utility occupancy of (ac) in T_1 is calculated as $uo(ac, T_1) = \$58/\$65 \approx 0.8923$, and the utility occupancy of (ac) in T_3 is calculated as $uo(ac, T_3) = \$32/\$38 \approx 0.8421$.

Definition 5: The utility occupancy of an itemset X in a database D is denoted as $uo(X)$, and defined as

$$uo(X) = \frac{\sum_{X \subseteq T_q \wedge T_q \in D} uo(X, T_q)}{|\Gamma_X|} \tag{2}$$

where Γ_X is the set of *supporting transactions* of X in D (thus $|\Gamma_X|$ is equal to the support of X in D).

Hence, the utility occupancy can be used to evaluate how patterns contribute to the total utility of transactions where they appears in. In prior work [18], the occupancy of a pattern in a transaction was defined as the ratio between the number of items in this pattern and the number of items in the transaction. The utility occupancy of a pattern generalizes this definition to consider that items may have distinct utility values, and is defined as the harmonic average of the occupancy utility values in all supporting transactions. Therefore, the two concepts, occupancy and utility occupancy, are different in nature.

Definition 6: Given a minimum support threshold α ($0 < \alpha \leq 1$) and a minimum utility occupancy threshold β ($0 < \beta \leq 1$), an itemset X in a database D is said to be an HUOP with high frequency and strong utility occupancy, denoted as HUOP, if it satisfies the following two conditions: $sup(X) \geq \alpha \times |D|$ and $uo(X) \geq \beta$.

Example 4: The utility occupancies of (a) and (ab) in Table I are calculated as: $uo(a) = (uo(a, T_1) + uo(a, T_3) + uo(a, T_5) + uo(a, T_8) + uo(a, T_9))/5 = (0.2154 + 0.5526 + 0.1429 + 0.3443 + 0.2373)/5 \approx 0.2985$; and $uo(ab) = (uo(ab, T_3) + uo(ab, T_5) + uo(ab, T_8))/3 = (0.6579 + 0.2653 + 0.3371)/3 = 0.4201$. When α and β are set to 30% and 0.30, the complete set of HUOPs in the running example database is: (c), (e), (ab), (ac), (ad), (bc), (bd), (cd), (ce), (abd), and (acd), as shown in Table II. Clearly, the utility occupancy measure does not respect the *downward closure* property of *Apriori* in FPM.

Note that the minimum support count must be set such that $\alpha \times |D| > 1$. Otherwise, each transaction (or an itemset) will have a utility occupancy value of 1. In the example of Table I, if α is set to 0.1, then $(\alpha \times |D|) = 1$, and all ten transactions are HUOPs. For example, T_1 [i.e., the itemset (acd)] has a support count as 1 and a utility occupancy as 1.0, thus satisfying the two conditions of HUOP. Based on the above definitions, the problem statement of HUOPM is formulated as follows.

TABLE II
DERIVED HUOPS

Pattern	*sup*	*uo*	Pattern	*sup*	*uo*
(c)	8	0.6468	(bd)	4	0.3620
(e)	4	0.4022	(cd)	5	0.6881
(ab)	3	0.4334	(ce)	3	0.8776
(ac)	4	0.8273	(abd)	3	0.4959
(ad)	5	0.3609	(acd)	4	0.8972
(bc)	3	0.6554			

Problem Statement: Given a transaction database D, a profit-table indicating the distinct profit of each item, a minimum support threshold α, and a minimum utility occupancy threshold β. The problem of mining HUOPs (which are both frequent and dominant in terms of high-utility occupancy) is to discover the complete set of patterns that not only have a frequency no less than $\alpha \times |D|$ but also have a utility occupancy no less than β.

IV. Proposed Algorithm for Mining HUOPs

In the section, we propose two compact data structures called utility-occupancy list (UO-list) and frequency-utility table (FU-table) to maintain the utility occupancy information about a database. Then, we utilize both the support and utility occupancy measures to prune the search space for mining the more interesting and useful HUOPs.

A. Search Space for Mining HUOPs

According to the previous studies, the search space for the pattern mining problem can be represented as a lattice structure [48] or as a set-enumeration tree [49]. Based on the final derived HUOPs of the given example, it can be observed that the well-known *downward closure* property of *Apriori* does not hold for HUOPs. For example, 1-items (a), (b), and (d) are not HUOPs, but their supersets $\{(ab), (ac), (ad), (bc), (bd), (cd), (abd), (acd)\}$ are HUOPs, as shown in Table II. If no anti-monotone property is applied in HUOPM, a huge number of candidates will need to be generated to obtain the actual HUOPs. It is thus a critical issue to design more suitable data structures and powerful pruning strategies to efficiently reduce the search space and to filter the number of unpromising patterns for mining HUOPs.

Definition 7 (Total Order $\prec$ On Items): Without loss of generality, assume that items in every transaction are sorted according to the lexicographic order. Furthermore, assume that the total order $\prec$ on items in the designed HUOPM algorithm adopts the support ascending order of items.

Note that the total order $\prec$ used in the proposed HUOPM algorithm can be the support-based ascending or descending order, the TWU-based ascending or descending order, the lexicographic order, or any other total order on items. The processing order of patterns may, however, affect the mining efficiency of HUOPM, but does not affect the completeness and correctness of this algorithm. We assume that the support ascending order is adopted in the HUOPM algorithm. Notice that the efficiency when using different orders in HUOPM

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

will be conducted and evaluated in the experiments. For example, the support counts of items in the running example are {*sup*(*a*): 5, *sup*(*b*): 5, *sup*(*c*): 8, *sup*(*d*): 7, *sup*(*e*): 4}. Thus, the support ascending order for five items is $\{sup(e) \prec sup(a) \leq sup(b) \prec sup(d) \prec sup(c)\}$, and the total order $\prec$ on items is $e \prec a \prec b \prec d \prec c$.

Definition 8 (FU-Tree): An frequency-utility tree (FU-tree) is a variant of sorted set-enumeration tree using the total order $\prec$ on items. It contains the frequency and utility information.

Definition 9 (Extension Nodes in the FU-Tree): In the designed FU-tree, all child nodes of any tree node are called its extension nodes.

Clearly, the designed FU-tree is a utility-based prefix-tree, and the complete search space can be traversed using a depth-first search (DFS) or breadth-first search, where each child node in the FU-tree is generated by extending its prefix (parent) node. Consider the running example with the adopted total order $\prec$ on items, the extension nodes of node (*ab*) are the itemsets (*abd*), (*abc*), and (*abdc*). Note that all the supersets of node (*ab*) are (*eab*), (*abd*), (*abc*), (*eabd*), (*eabc*), (*abde*), and (*eabde*). Hence, the extension nodes of a tree node in an FU-tree are a subset of the supersets of that node.

B. UO-List and FU-Table

Two compact data structures, called utility-occupancy list (UO-list for short) and frequency-utility table (FU-table for short), are designed to keep essential information about patterns in the database. Note that the UO-list is different from the utility-list structure used in HUI-Miner [12], FHM [30], and OCEAN [19]. The occupancy information is not stored in utility-list, while UO-list keeps this information to quickly calculate the utility occupancy of a pattern using a join operation. Besides, a new concept called *remaining utility occupancy* is introduced and applied to obtain an utility occupancy upper bound, which will be presented in the next section.

Definition 10: The remaining utility occupancy of an itemset X in a transaction T_q is denoted as $ruo(X, T_q)$, and defined as the sum of the utility occupancy values of all items appearing after X in T_q according to the total order $\prec$, that is

$$ruo(X, T_q) = \frac{\sum_{i_j \notin X \wedge X \subseteq T_q \wedge X \prec i_j} u(i_j, T_q)}{tu(T_q)}. \quad (3)$$

For example, consider T_5 and itemsets (*a*) and (*ad*) in Table I. Since the total order $\prec$ on items is $e \prec a \prec b \prec d \prec c$, we have that $ruo(a, T_5) = (u(b, T_5) + u(d, T_5) + u(c, T_5))/tu(T_5) = (\$6 + \$5 + \$22)/\$49 \approx 0.6735$, and $ruo(ad, T_5) = (u(c, T_5))/tu(T_5) = \$22/\$49 \approx 0.4490$.

Definition 11 (UO-List): Let $\prec$ be the total order on items from I. The UO-list of an itemset X in a database D is a set of tuples corresponding to transactions where X appears. A tuple contains three elements (tid, uo, ruo) for each transaction T_q containing X. In each tuple, the *tid* element is the transaction identifier of T_q; the *uo* element is the utility occupancy of X in T_q, with respect to $uo(X, T_q)$; and the *ruo* element is defined as the remaining utility occupancy of X in T_q, with respect to $ruo(X, T_q)$.

Example 5: Consider the running example and the defined total order $\prec$ ($\{e \prec a \prec b \prec d \prec c\}$). The constructed UO-lists of five 1-itemsets (itemsets of length 1) are shown in Fig. 1. Note that the UO-lists of all 1-itemsets are constructed after the HUOPM algorithm performs a single database scan.

(*e*)		
tid	***uo***	***ruo***
5	0.1837	0.8163
6	0.6207	0.3793
8	0.5902	0.4098
10	0.2143	0.7857

(*a*)		
tid	***uo***	***ruo***
1	0.2154	0.7846
3	0.5526	0.4474
5	0.1429	0.6735
8	0.3443	0.0656
9	0.2373	0.7627

(*c*)		
tid	***uo***	***ruo***
1	0.6769	0
2	0.8919	0
3	0.2895	0
5	0.4490	0
6	0.3793	0
7	0.9565	0
9	0.7458	0
10	0.7857	0

Fig. 1. Constructed UO-lists of the five items.

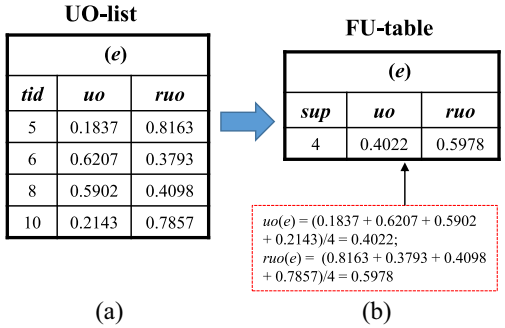


Fig. 2. UO-list and FU-table of item (*e*).

To discover HUOPs, the sum of the support and utility occupancy of a special pattern X in a database D can be efficiently calculated by adding the utility occupancies of all elements in the UO-list of X (denoted as $X.UOL$). Thus, the following information can be obtained from $X.UOL$: 1) the name of X; 2) the set of transactions where X appears (its support); 3) the sum of the utility occupancy of X in D; and 4) the total remaining utility occupancy of X in D. To explain how this useful information can be obtained from the constructed UO-lists, the following definitions are introduced by utilizing the UO-list structure. We further design a data structure called FU-table by utilizing some useful properties of UO-list. The FU-table of an expected pattern is built after the construction of UO-list of this pattern, and it stores the following information.

Definition 12 (FU-Table): An frequency-utility table (FU-table) of an itemset X contains four information: 1) the name of the itemset X (*name*); 2) the support of X ($sup(X)$); 3) the sum of the utility occupancies of X in database D ($uo(X)$); and 4) the sum of the remaining utility occupancies of X in D ($ruo(X)$). Here, $ruo(X)$ can be calculated as

$$ruo(X) = \frac{\sum_{X \subseteq T_q \wedge T_q \in D} ruo(X, T_q)}{|\Gamma_X|}. \quad (4)$$

The construction process of an FU-table is shown below. Consider an item (*e*) in Table I, which appears in T_5, T_6, T_8, and T_{10}. The built UO-list of (*e*) is shown in Fig. 2(a). The FU-table of item (*e*) is constructed efficiently by using the support count, utility occupancy, and remaining utility occupancy. They are calculated during the construction of the UO-list of (*e*), such that $\{sup(e) = 4, uo(e) = (0.1837 + 0.6207 + 0.5902 + 0.2143)/4 = 0.4022$, and $ruo(e) =$

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

(e)		
sup	*uo*	*ruo*
4	0.4022	0.5978

(a)		
sup	*uo*	*ruo*
5	0.2985	0.5467

(b)		
sup	*uo*	*ruo*
5	0.2192	0.4181

(d)		
sup	*uo*	*ruo*
7	0.0898	0.4454

(c)		
sup	*uo*	*ruo*
8	0.6468	0

Fig. 3. Constructed FU-tables of all 1-itemsets.

Algorithm 1 Construction (X, X_a, X_b)

```
1: set X_ab.UOL ← ∅, X_ab.FUT ← ∅;
2: for each tuple E_a ∈ X_a.UOL do
3:   if ∃E_a ∈ X_b.UOL ∧ E_a.tid == E_b.tid then
4:     if X.UOL ≠ ∅ then
5:       search for E ∈ X.UOL, E.tid = E_a.tid;
6:       E_ab ← < E_a.tid, E_a.uo + E_b.uo - E.uo, E_b.ruo >;
7:       X_ab.FUT.uo += E_a.uo + E_b.uo − E.uo;
8:       X_ab.FUT.ruo += E_b.ruo;
9:     else
10:      E_ab ←< E_a.tid, E_a.uo + E_b.uo, E_b.ruo >;
11:      X_ab.FUT.uo += E_a.uo + E_b.uo;
12:      X_ab.FUT.ruo += E_b.ruo;
13:    end if
14:    X_ab.UOL ← X_ab.UOL ∪ E_ab;
15:    X_ab.FUT.sup ++;
16:  else
17:    X_a.FUT.sup - -;
18:    if X_a.FUT.sup < α × |D| then
19:      return null;
20:    end if
21:  end if
22: end for
23: return X_ab
```

$(0.8163 + 0.3793 + 0.4098 + 0.7857)/4 = 0.5978\}$, and the results of its FU-table are $\{sup(e) = 4, uo(e) = 0.4022$, and $ruo(e) = 0.5978\}$, as shown in Fig. 2(b). Thus, the final built FU-tables of all items (1-itemsets) are shown in Fig. 3.

According to the two conditions $sup(X) \geq \alpha \times |D|$ and $uo(X) \geq \beta$ of HUOP, we only need to initially construct the UO-lists and FU-tables of frequent 1-itemsets (FP^1). Then, for any k-itemset ($k \geq 2$), the UO-list of this itemset can be calculated directly using the UO-lists of some of its subsets, without scanning the database. The construction procedure of the UO-list and FU-table of a k-itemset is shown in Algorithm 1. Given an itemset X, and two itemsets X_a and X_b which are extensions of X obtained by adding two distinct items a and b to X, respectively. The construction procedure takes as input the UO-lists of X, X_a, and X_b, and outputs the UO-list and FU-table of the itemset X_{ab}. Note that $X_{ab}.UOL$ denotes the UO-list of X_{ab}, and $X_{ab}.FUT$ denotes the FU-table of X_{ab}.

In Algorithm 1, lines 17–20 show the developed pruning strategy named *remaining support strategy*, which will be described later. It is important to notice that the construction of the UO-list and FU-table of a k-itemset ($k \geq 3$, lines 5–8) is different than that of a 2-itemset ($k = 2$, lines 10–12). Details of the difference are described as the following contents. The uo of X_{ab} is the sum of the uo associated with tid in the UO-lists of X_a and X_b. Suppose a is before b, and then the ruo of X_{ab} is assigned as the ruo associated with tid in the UO-list of X_b. However, when calculating the uo and ruo for a (k)-itemset ($k \geq 2$), the part of uo is different, thus it needs to subtract the uo of the common part between X_a and X_b. Generally, to calculate the uo of $\{i_1 \dots i_{(k-2)} i_{(k-1)} i_k\}$ in tid, the following formula holds: $uo(\{i_1 i_{(k-2)} i_{(k-1)} i_k\}, tid) = uo(\{i_1 \dots i_{(k-2)} i_{(k-1)}\}, tid) + uo(\{i_1 \dots i_{(k-2)} i_k\}, tid) - uo(\{i_1 \dots i_{(k-2)}\}, tid)$. For example, in transaction T_5, $uo(ac, T_5) = uo(a, T_5) + uo(c, T_5) = 0.1429 + 0.4490 = 0.5919$, $uo(ae, T_5) = uo(a, T_5) + uo(e, T_5) = 0.1429 + 0.1837 = 0.3266$, and $uo(ace, T_5) = uo(ac, T_5) + uo(ae, T_5) - uo(a, T_5) = 0.5919 + 0.3266 - 0.1429 = 0.7756$. We cannot miscalculate the uo of $\{ace\}$ in T_5 such as the sum of the uo of $\{ac\}$ and $\{ae\}$ in T_3, since they contain the uo of $\{a\}$ in T_5 twofold.

The procedure can be easily implemented since a set of the UO-lists of the $(k-1)$-itemsets ($k \geq 2$) has been built before constructing the UO-list of a (k)-itemset ($k \geq 2$) with respect to the extension node.

C. Upper Bound on Utility Occupancy

Using the proposed UO-list and FU-table structures, the actual utility occupancy value of a pattern can be calculated exactly. But a crucial question is: is it necessary to construct the UO-lists and FU-tables of all patterns in an FU-tree? Since this would be very expensive in terms of runtime and memory usage, some pruning conditions should be developed to decide whether a subtree must be spanned. In a set-enumeration tree [49], the complete search space of I (where m is the number of items in I) contains 2^m patterns (by systematically enumerating all subsets of I with the total order $\prec$, that is all possible patterns). As mentioned above, the *downward closure* property does not hold for the utility occupancy. Thus, it is a crucial challenge to design powerful technologies to prune the search space and filter unpromising patterns early, especially to handle large-scale databases.

Without actually generating all possible HUOPs, can we derive an upper bound $\hat{\phi}(X)$ on the utility occupancy of patterns in a subtree rooted at node representing an itemset X? Unfortunately, it is not an easy task since the utility occupancy and the supporting transactions of patterns are unknown until these patterns are processed. Hence, inspired by the OCEAN algorithm [19], we try to develop an upper bound on the utility occupancy by utilizing the UO-list structure. If $\hat{\phi}(X)$ is less than the minimum utility occupancy threshold, we can safely discard this subtree from further consideration.

Definition 13: Let there be an itemset X and a transaction (or an itemset) T such that $X \subseteq T$, and that the set of all items in T that are not in X is denoted as $T \setminus X$. In addition, the set of all items appearing after X in T according to the $\prec$ order is denoted as T/X. Thus, $T/X \subseteq T \setminus X$.

Lemma 1: Let there be a subtree rooted at X, Γ_X be the supporting transactions of X. Then, for any possible high-utility occupancy itemset W in the subtree, we have

$$uo(W) \leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D} \big(uo(X, T_q) + ruo(X, T_q)\big)}{|\Gamma_W|}. \quad (5)$$

Proof: Since W is a k-extension of X, $\Gamma_W \subseteq \Gamma_X$, we have $(W - X) = (W/X)$. Thus, in each transaction T_q,

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

$uo(W/X, T_q) \leq uo(T_q/X, T_q)$. The following relationships can be obtained. For each transaction T_q in D, since

$$\begin{aligned} uo(W) &= \frac{\sum_{W \subseteq T_q \wedge T_q \in D} uo(X, T_q)}{|\Gamma_W|} \\ &= \frac{\sum_{W \subseteq T_q \wedge T_q \in D} (uo(X, T_q) + uo(W - X, T_q))}{|\Gamma_W|} \\ &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D} (uo(X, T_q) + uo(T_q/X, T_q))}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D} (uo(X, T_q) + uo(T_q/X, T_q))}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D} (uo(X, T_q) + ruo(X, T_q))}{|\Gamma_W|}. \end{aligned}$$

■

The set of supporting transactions of a processed node (itemset) X is denoted as Γ_X, but the set of supporting transactions of any possible HUOP W extending X with respect to Γ_W, is unknown although $\Gamma_W \subseteq \Gamma_X$. The set Γ_W is unknown until W is processed. Assume that we now explore the node (ea). After constructing the UO-list of (ea), we can obtain its support count and utility occupancy. Unfortunately, we do not know the related support count Γ_W of any of its extension nodes W. Because of inequality (5), it is difficult to obtain an upper bound for W without knowing Γ_W. The concept of HUOP indicates that every HUOP should be supported by $\alpha \times |D|$ transactions in the database. By utilizing this property, we further develop the following theorems to obtain an upper bound for a subtree rooted at a processed node in the FU-tree.

Lemma 2: Let there be a minimum support threshold α, a subtree rooted at X, and Γ_X be its supporting transactions. For any pattern W in the subtree, an upper bound on the utility occupancy of W is

$$\hat{\phi}(W) = \frac{\sum_{\text{top}\alpha \times |D|, T_q \in \Gamma_X} \big(uo\big(X, T_q\big) + ruo\big(X, T_q\big)\big)}{|\alpha \times |D||} \quad (6)$$

$$\hat{\phi}(W) \geq uo(W). \quad (7)$$

Proof: Note that $\Gamma_W \subseteq \Gamma_X$ and Γ_W is unknown, we calculate $(uo(X, T_q) + ruo(X, T_q))$ for all transactions in Γ_X and put into a vector set (denoted as V_{occu}), then sort V_{occu} in descending order (denoted as $V^{\downarrow}_{\text{occu}}$). Since the average of top-k $(0 < k \leq |\Gamma_W|)$ values of vector $V^{\downarrow}_{\text{occu}}$ is an upper bound of the average of total $|\Gamma_W|$ values of $[(\sum_{W \subseteq T_q \wedge T_q \in D}(uo(X, T_q) + ruo(X, T_q)))/|\Gamma_W|]$. Since an HUOP should be supported by at least $\alpha \times |D|$ transactions, we have $\alpha \times |D| \leq k \leq |\Gamma_W| \leq |\Gamma_X|$. We have

$$\begin{aligned} uo(W) &\leq \frac{\sum_{W \subseteq T_q \wedge T_q \in D} (uo(X, T_q) + ruo(X, T_q))}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{\text{top}k, T_q \in \Gamma_X} \{uo(X, T_q) + ruo(X, T_q)\}^{\downarrow}}{|\Gamma_W|} \\ \Longrightarrow uo(W) &\leq \frac{\sum_{\text{top}\alpha \times |D|, T_q \in \Gamma_X} \{uo(X, T_q) + ruo(X, T_q)\}^{\downarrow}}{|\alpha \times |D||} \\ \Longrightarrow uo(W) &\leq \hat{\phi}(W). \end{aligned}$$

Thus, given a minimum support threshold α, we can directly calculate an upper bound $\hat{\phi}(W)$ on utility occupancy of a subtree which rooted at a processed node X. ■

D. Proposed Pruning Strategies

In this section, we will show how to efficiently prune the search space using an upper bound on the utility occupancy and the support count. Two properties named *global downward closure* property and *partial downward closure* property can be obtained below.

Lemma 3: The complete search space of the addressed HUOPM problem can be represented by an FU-tree where items are sorted according to the support ascending order on items.

Proof: According to [49], the complete search space for mining HUOPs can be presented as a set-enumeration tree.

Theorem 1 (Global Downward Closure Property in the FU-Tree): In the designed FU-tree, if a tree node is an FP, its parent node is also an FP. Let X^k be a k-itemset (node) and its parent node be denoted as X^{k-1}, which is a $(k-1)$-itemset. The relationship $sup(X^k) \leq sup(X^{k-1})$ holds.

Proof: According to the well-known *Apriori* property [1], the relationship $sup(X^k) \leq sup(X^{k-1})$ exists. Thus, in the FU-tree, the *global downward closure* property holds. ■

Theorem 2 (Partial Downward Closure Property in the FU-Tree): In the designed FU-tree, let X^k be a k-itemset (node) and its parent node be denoted as X^{k-1}, a $(k-1)$-itemset. The relationship indicating that the upper bound on utility occupancy of any node in a subtree is no greater than that of its parent node always holds, that is, $\hat{\phi}(X^k) \leq \hat{\phi}(X^{k-1})$.

Proof: Since the UO-list of X^k is constructed by joining the UO-list of X^{k-1} with one of the siblings of X^{k-1}, we have $\Gamma_{X^k} \subseteq \Gamma_{X^{k-1}}$. The average of the top-k $(0 < k \leq |\Gamma_{X^k}|$ values in the vector $V^{\downarrow}_{\text{occu}}$ of X^k is no greater than the average of top-k $(0 < k \leq |\Gamma_{X^{k-1}}|$ values of vector $V^{\downarrow}_{\text{occu}}$ of X^{k-1}. According to Lemma 2, the relationship between the upper bounds on the utility occupancy of X^k and X^{k-1} is: $\hat{\phi}(X^k) \leq \hat{\phi}(X^{k-1})$. As a result, if $\hat{\phi}(X^k) \geq \beta$, then $\hat{\phi}(X^{k-1}) \geq \hat{\phi}(X^k) \geq \beta$. Conversely, if $\hat{\phi}(X^{k-1}) < \beta$, then $\hat{\phi}(X^k) \leq \hat{\phi}(X^{k-1}) < \beta$. In other words, the obtained upper bound on the utility occupancy of patterns satisfies the *partial downward closure* property. Thus, in the developed FU-tree, the *partial downward closure* property holds. ■

It is noteworthy that *partial* here indicates that the upper bound on the utility occupancy is conditional anti-monotone but not general anti-monotone. In other words, the *partial downward closure* property holds for a tree node and its descendants, but does not hold for supersets which are not in that subtree. To further enhance the pruning effect of the proposed HUOPM algorithm, the derived upper bound can be utilized with this anti-monotone property. We next present four novel pruning strategies, which were not used in the state-of-the-art OCEAN algorithm [19].

Strategy 1: In the designed FU-tree and by considering the defined total order $\prec$, if a tree node X has a support count less than $(\alpha \times |D|)$, then any nodes containing X (i.e., all supersets of X) can be directly pruned and do not need to be explored.

Example 6: In the running example, assume that the parameters are set to $\alpha = 30\%$ and $\beta = 0.3$. Using the UO-lists

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 2 HUOPM (D, *ptable*, α, β)

1: scan D to calculate the $sup(i)$ of each item $i \in I$ and the tu value of each transaction;
2: find $I^* \leftarrow \{i \in I | sup(i) \geq \alpha \times |D|\}$, w.r.t. FP^1;
3: sort I^* in the designed total order $\prec$;
4: using the total order $\prec$, scan D once to build the UO-list and FU-table for each 1-item $i \in I^*$;
5: **call *HUOP-Search***$(\phi, I^*, \alpha, \beta)$.
6: **return** *HUOPs*

Algorithm 3 HUOP-Search (X, *extenOfX*, α, β)

1: **for** each itemset $X_a \in extenOfX$ **do**
2: obtain $sup(X_a)$ and $uo(X_a)$ from the built $X_a.FUT$;
3: **if** $sup(X_a) \geq \alpha \times |D|$ **then**
4: **if** $uo(X_a) \geq \beta$ **then**
5: $HUOPs \leftarrow HUOPs \cup X_a$;
6: **end if**
7: $\hat{\phi}(X_a) \leftarrow$ ***UpperBound***$(X_a.UOL, \alpha)$;
8: **if** $\hat{\phi}(X_a) \geq \beta$ **then**
9: $extenOfX_a \leftarrow \emptyset$;
10: **for** each $X_b \in extenOfX$ that $X_a \prec X_b$ **do**
11: $X_{ab} \leftarrow X_a \cup X_b$;
12: call ***Construct***(X, X_a, X_b);
13: **if** $X_{ab}.UOL \neq \emptyset \wedge sup(X_{ab}) \geq \alpha \times |D|$ **then**
14: $extenOfX_a \leftarrow extenOfX_a \cup X_{ab}.UOL$;
15: **end if**
16: **end for**
17: **call *HUOP-Search***$(X_a, extenOfX_a, \alpha, \beta)$;
18: **end if**
19: **end if**
20: **end for**
21: **return** *HUOPs*

of (e) and (a), we can construct the UO-list of (ea). Since $sup(ea) = 2 < (\alpha \times 10 = 30\% \times 10 = 3)$, all extension nodes of (ea) are not HUOPs and can be directly pruned.

Strategy 2: In the designed FU-tree, considering the defined total order $\prec$, if the upper bound on the utility occupancy of a tree node X is less than β, then any nodes in the subtree rooted at X with respect to all extensions of X can be directly pruned and do not need to be explored.

Example 7: After constructing the UO-list of (ab), we can obtain its utility occupancy (uo) and remaining utility occupancy (ruo) in each supporting transaction. According to inequality (5), the top-three values in the sorted vector $V^{\downarrow}_{occu}$ of (ab) are considered. Since the UO-list of (ab) is $\{(T_3, 0.6579, 0.3421), (T_5, 0.2653, 0.5510), (T_8, 0.3771, 0.0328)\}$, $V^{\downarrow}_{occu}$ of $(db) = \{(T_3, 1.0000), (T_5, 0.8163), (T_8, 0.4099)\}$. Thus, the utility occupancy upper bound of (ab) is $(1.0000 + 0.5380 + 0.4099)/3 \approx 0.7421 > 0.3$, and $sup(ab) = 3$. Thus, its extension nodes (abd), (abc), and $(abdc)$ may be HUOPs and, hence, should be explored.

Strategy 3: During the construction of the UO-list of a tree node X_{ab} by using X_a and X_b, if the remaining support of X_a for constructing X_{ab} is less than $\alpha \times |D|$, the support of X_{ab} will also be less than $\alpha \times |D|$, X_{ab} is not an FP, and also not an HUOP. Then, the construction procedure returns *null*, as shown in Algorithm 1 (lines 18–20).

Strategy 4: After calling the construction procedure to build the UO-list of a tree node X, if $X.UOL$ is empty or $sup(X) \leq \alpha \times |D|$, X is not an HUOP, and none of its subtree nodes is an HUOP. Then, $X.UOL$ is not added to the set of extension UO-lists of X.

Example 8: In the running example, the node (ea) has a support count of 2. Its subtree nodes can be pruned by Strategy 4. In this case, Strategy 4 has the same effect as Strategy 1.

E. Proposed HUOPM Algorithm

Based on two compact data structures (UO-list and FU-table), an upper bound on utility occupancy and the above pruning strategies, the proposed HUOPM algorithm can be designed below. Algorithm 2 shows the pseudocode of the designed HUOPM algorithm. It takes four parameters as input, including a transactional database D, a predefined profit-table *ptable*, a user-specified minimum support threshold α $(0 < \alpha \leq 1)$, and a minimum utility occupancy threshold β $(0 < \beta \leq 1)$. HUOPM first scans the database to calculate the $sup(i)$ of each item $i \in I$ and the tu value of each transaction (line 1), then finds the set I^* with respect to FP^1 (line 2, Strategy 1), and sorts it according to the total order $\prec$ (line 3). After that, the algorithm scans D again to construct the UO-list and FU-table of each item $i \in I^*$ (line 4). At last, HUOPM recursively applies the designed **HUOP-Search** function for mining HUOPs without generating candidates and scanning the database repeatedly (line 5).

During the recursive exploration of the search space (i.e., FU-tree) with the total order $\prec$, the partial anti-monotonicity of HUOPs is used to effectively avoid generating unpromising patterns that cannot be HUOPs and their child nodes. Based on the proposed pruning strategies, the upper bound can be used to prune patterns with low support or low utility occupancy early, without constructing their UO-lists. It can effectively reduce both the computational cost of join operations and the search space in the FU-tree. In fact, the actual search space is more compact than a complete FU-tree.

Details of the **HUOP-Search** procedure is shown in Algorithm 3. It takes as input an itemset X, a set of UO-lists of all 1-extensions of X, α, and β. A loop is first performed over each 1-extension of itemset X, denoted as X_a (lines 1–20). Two conditions are checked (lines 3 and 4, Strategy 1) to determine if X_a is an HUOP. If the itemset is an HUOP, then it is put into the set of HUOPs. The algorithm then determines whether extensions of X_a should be explored (lines 7 and 8, Strategy 2). It uses the function **UpperBound** (see Algorithm 4) to calculate the upper bound of utility occupancy for the subtree rooted at pattern X_a. If $\hat{\phi}(X_a) \geq \beta$, then any extensions of X_a may be an HUOP. In that case, the DFS is performed (lines 8–18). The construction procedure **Construct**(X, X_a, X_b) is then executed for each 1-extension node X_{ab}, to construct their UO-lists and FU-tables (lines 11 and 12). Note that each constructed itemset X_{ab} is an 1-extension of the itemset X_a. If $X_{ab}.UOL \neq \emptyset$ and $sup(X_{ab}) \geq \alpha \times |D|$, such itemset is added to a set $extenOfX_a$ for storing all 1-extensions of X_a (lines 13–15, Strategy 4). HUOPM recursively applies the

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 4 UpperBound ($X_q.UOL$, α)

1: $sumTopK \leftarrow 0, \hat{\phi}(X_a) \leftarrow 0, V_{occu} \leftarrow \emptyset$;
2: calculate $(uo(X, T_q) + ruo(X, T_q))$ of each tuple from the built $X_a.UOL$ and put them into the set of V_{occu};
3: sort V_{occu} by descending order as $V^{\downarrow}_{occu}$;
4: **for** $k \leftarrow 1$ to $\alpha \times |D|$ in $V^{\downarrow}_{occu}$ **do**
5: $sumTopK \leftarrow sumTopK + V^{\downarrow}_{occu}[k]$;
6: **end for**
7: $\hat{\phi}(X_a) = \dfrac{sumTopK}{\alpha \times |D|}$.
8: **return** $\hat{\phi}(X_a)$

designed HUOP-Search function for determining the desired HUOPs (line 17).

Algorithm 4 provides the pseudocode of **UpperBound** procedure, for computing the upper bound of utility occupancy. The complexity of Algorithm 4 is $O(n \times log(n) \times V^{\downarrow}_{\text{occu}}[k])$ (the top-k value in vector is equal to $\alpha \times |D|$). We first use $O(n \times log(n))$ time for sorting V_{occu} with quick sort algorithm in line 3, then use $O(n \times log(n) \times V^{\downarrow}_{\text{occu}}[k])$ time for the loop operation from lines 4–7 to calculate the top-k values in $V^{\downarrow}_{\text{occu}}[k])$ (lines 4 and 5). Finally, it returns the average value of the summation as the upper bound on utility occupancy of the subtree rooted at pattern X_a (lines 4 and 5).

V. Experiments

In this section, extensive experiments are conducted on both real-world and synthetic datasets to evaluate the effectiveness and efficiency of the proposed HUOPM algorithm. The task of the addressed problem is to discover HUOPs with frequency and utility occupancy. Note that only two prior studies, DOFIA [18] and OCEAN [19] are closely related to this paper. Major differences are that DOFIA considers both the frequency and occupancy for mining high qualified patterns, but the occupancy is based on the number of items in patterns or transactions rather than the concept of utility [18]. Thus, in the following, the state-of-the-art OCEAN algorithm [19] is implemented to generate HUOPs. HUOPs* are derived by OCEAN, and HUOPs are generated by the proposed HUOPM algorithm.

We first evaluate the mining results by comparing HUOPs* and HUOPs. Then, the runtime and the number of visited nodes in the search tree of the HUOPM algorithm are compared. At last, the effect of the processing order of items is also compared and evaluated. Note that in the following each figure, the subfigures (a) to (f) are related to BMSPOS2, retail, chess, mushroom, T10I4D100K and T40I10D100K, respectively.

A. Experimental Setup and Datasets

All algorithms in the experiments are implemented using the Java language and executed on a PC with an Intel Core i5-3460 3.2-GHz processor and 4 GB of memory, running on the 32 bit Microsoft Windows 7 platform. Four real-world datasets [50] (BMSPOS2, retail, chess, and mushroom) and two synthetic dataset [51] (T10I4D100K and T40I10D100K) are used in the experiments. T10I4D100K and T40I10D100K are generated using the IBM Quest Synthetic Data Generator [51]. These datasets have varied characteristics and represent the main types of data typically encountered in real-life scenarios (e.g., dense, sparse, and long transactions). The characteristics of these datasets are described below in details.

1) *BMSPOS2* dataset contains several years worth of point-of-sale data from a large electronics retailer. It has total 515 597 transactions with 1657 distinct items, an average transaction length of 6.53 items.
2) *retail* is a sparse dataset which contains 88 162 transactions with 16 470 distinct items and an average transaction length of 10.30 items.
3) *chess* dataset contains 3196 transactions with 75 distinct items and an average transaction length of 36 items. It is a dense dataset.
4) *mushroom* is a dense dataset, it has 812 transactions with 120 distinct items, and an average transaction length of 23 items.
5) *T10I4D100K* is a synthetic dataset that contains 870 distinct items, 100 000 transactions, and has an average length of 10.1 items.
6) *T40I10D100K* is a synthetic dataset that contains 942 distinct items, 100 000 transactions, and has an average length of 39.6 items.

For the addressed utility-based HUOPM problem, note that the quantity and unit profit of each item in a dataset is randomly generated by using a simulation method proposed in the previous studies [12]–[14]. In the following experiments, the proposed HUOPM algorithm with different designed pruning strategies is, respectively, denoted as HUOPM$_{P12}$ (with pruning Strategies 1 and 2); HUOPM$_{P13}$ (with pruning Strategies 1 and 3); HUOPM$_{P123}$ (with pruning Strategies 1–3); and HUOPM$_{P1234}$ (with all pruning strategies). The four versions are compared to evaluate the efficiency of HUOPM and the effect of the designed pruning strategies.

B. Pattern Analysis

In the study presenting OCEAN, it was shown that the diversity of patterns with high-utility occupancy is more favorable than traditional HUPs and FPs. The study used a small real-life dataset *MobileApp* to illustrate that the derived patterns are more useful for the task of mobile App recommendation or promotion [19]. To analyze the usefulness of the proposed HUOPM framework, the derived patterns (HUOPs* and HUOPs) are evaluated below. When setting α:20% and β:0.50 on the running example, the top-10 patterns derived by HUOPM, OCEAN, and FP-growth [2] are shown in Table III, respectively. Notice that the results in left, middle, and right of Table III are sorted by the descending order of utility occupancy (*uo*) and support (*sup*), respectively. It is clear that three real HUOPs of top-10 results [(*abde*), (*abe*), and (*cd*), as shown in Table III (left)] are missed in the results of OCEAN [as shown in Table III (middle)]. Besides, the minimum *utility occupancy* in top-10 HUOPs* is 0.5604, while the minimum *utility occupancy* value in top-10 HUOPs is 0.6554. Some interesting desired patterns with high *uo* values cannot be discovered by OCEAN. Therefore, the results derived by the OCEAN algorithm are incomplete and OCEAN also encounters some performance problems (it will be discussed in the next section). Finally, it can be clearly seen that the support-based FPs [as shown in Table III (right)] are quite different from the results derived by the utility-occupancy-based methods.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE III
DERIVED PATTERNS FROM RUNNING EXAMPLE

(**left**) Top-10 HUOPs			(**middle**) Top-10 HUOPs*			(**right**) Top-10 FPs		
Pattern	***sup***	***uo*↓**	**Pattern**	***sup***	***uo*↓**	**Pattern**	***sup*↓**	***uo***
$(abcd)$	2	0.9081	$(abcd)$	2	0.9081	(c)	8	0.6468
(acd)	4	0.8972	(acd)	4	0.8972	(d)	7	0.0897
(ce)	3	0.8776	(ce)	3	0.8776	(a)	5	0.2985
(abc)	2	0.8308	(abc)	2	0.8308	(b)	5	0.2192
(ac)	4	0.8273	(ac)	4	0.8273	(ad)	5	0.3609
$(abde)$	2	0.7755	(ade)	2	0.6979	(cd)	5	0.6881
(abe)	2	0.7081	(bc)	3	0.6554	(e)	4	0.4022
(ade)	2	0.6979	(c)	8	0.6468	(ac)	4	0.8273
(cd)	5	0.6881	(ae)	2	0.6305	(bd)	4	0.3620
(bc)	3	0.6554	(bcd)	2	0.5604	(acd)	4	0.8972

TABLE IV
DERIVED PATTERNS UNDER VARIED α

Dataset	**Patterns**	**# patterns by varying threshold α**					
		α_1	α_2	α_3	α_4	α_5	α_6
BMSPOS2	HUOPs*	29,389	17,005	11,876	8,961	7,065	5,884
(β: 0.3)	HUOPs	29,444	17,048	11,899	8,987	7,094	5,905
retail	HUOPs*	12,801	8,155	5,902	4,513	3,989	3,282
(β: 0.3)	HUOPs	13,105	8,209	5,927	4,527	4,004	3,291
chess	HUOPs*	2,423	1,776	1,374	1,108	903	695
(β: 0.3)	HUOPs	11,469	8,949	7,075	5,510	4,273	3,351
mushroom	HUOPs*	17,678	12,429	10,739	10,252	2,821	1,199
(β: 0.3)	HUOPs	52,795	33,629	31,741	31,025	4,613	1,672
T10I4D100K	HUOPs*	536,414	246,932	138,877	91,557	67,834	53,957
(β: 0.3)	HUOPs	550,676	253,617	144,106	96,505	72,555	59,068
T40I10D100K	HUOPs*	33,171	14,415	10,465	258	151	55
(β: 0.3)	HUOPs	71,941	15,496	12,273	1,367	1,002	548

TABLE V
DERIVED PATTERNS UNDER VARIED β

Dataset	**Patterns**	**# patterns by varying threshold β**					
		β_1	β_2	β_3	β_4	β_5	β_6
BMSPOS2	HUOPs*	287,850	106,239	34,195	11,876	5,089	2,696
(α: 0.02%)	HUOPs	287,902	106,285	34,233	11,899	5,115	2,721
retail	HUOPs*	31,036	13,910	5,902	2,295	842	315
(α: 0.014%)	HUOPs	31,067	13,940	5,927	2,335	883	346
chess	HUOPs*	9,820	4,837	1,374	514	125	14
(α: 62%)	HUOPs	24,905	14,050	7,075	3,153	1,170	357
mushroom	HUOPs*	16,733	10,739	4,914	1,327	505	136
(α: 18%)	HUOPs	41,311	31,741	21,755	12,917	6,431	2,556
T10I4D100K	HUOPs*	181,541	131,631	91,557	61,079	38,980	23,435
(α: 0.012%)	HUOPs	182,538	135,710	96,505	66,772	44,380	27,906
T40I10D100K	HUOPs*	60,749	37,163	21,105	10,465	3,898	911
(α: 0.8%)	HUOPs	68,457	42,887	24,592	12,273	5,031	1,594

We further compare the number of patterns on the test datasets to show the effect of varying the parameters α and β. Results for various parameter values are shown in Tables IV and V, respectively. Notice that the parameters in Table IV are varying α under a fixed $\beta = 0.3$, and the parameter settings on each dataset are the same as them in Fig. 4. In Table V, the parameters are varying β under a fixed α, and the details of the parameter settings in each dataset are the same as them in Fig. 5.

From Tables IV and V, it can be clearly observed that the sets of derived patterns are quite different for various α or β values. The number of HUOPs* is always smaller than that of HUOPs. For example, on chess dataset as shown in Tables IV and V, most of the final HUOPs are missed by the OCEAN algorithm since the number of HUOPs* is always considerably smaller than that of HUOPs. It means that numerous

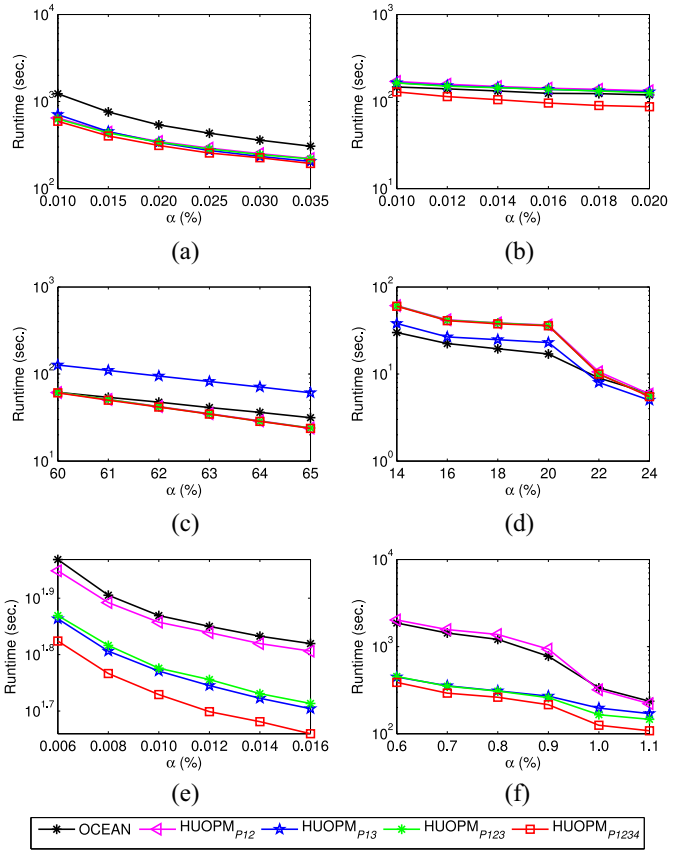


Fig. 4. Runtime under varied α with a fixed β. (a) BMSPOS2 (β:0.3). (b) retail (β:0.3). (c) chess (β:0.3). (d) mushroom (β:0.3). (e) T10I4D100K (β:0.3). (f) T40I10D100K (β:0.3).

interesting HUOPs are effectively discovered by HUOPM algorithm, while most of them are missed by the OCEAN algorithm. In other words, although the addressed HUOPM problem can capture high qualified patterns well, OCEAN fails to discover the complete set of HUOPs. It can also be observed that the number of produced patterns (both HUOPs* and HUOPs), decrease when the minimum support threshold is increased. And less HUOPs* and HUOPs are obtained when β is set higher. Besides, the number of missing patterns which is caused by OCEAN (i.e., HUOPs–HUOPs*) sometimes increases when varying β, while it sometimes decreases, as shown on the chess and T10I4D100K datasets when varying β with a fixed α. From the above analysis of the results of found patterns, it can be concluded that the proposed HUOPM algorithm for mining HUOPs is acceptable and can solve a serious shortcoming of the state-of-the-art OCEAN algorithm.

C. Efficiency Analysis

To evaluate the efficiency of OCEAN and the proposed HUOPM algorithm on various datasets, a performance comparison of the different strategies used in HUOPM is presented next in terms of execution time. The results in terms of runtime for various parameters are shown in Figs. 4 and 5, respectively.

From Figs. 4 and 5, it can be clearly observed that the runtime of OCEAN is the worst, compared to the other algorithms in most cases, and that the HUOPM$_{P1234}$ algorithm (which adopts all pruning strategies) is the fastest on all datasets in all cases. We also draw the following conclusions.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

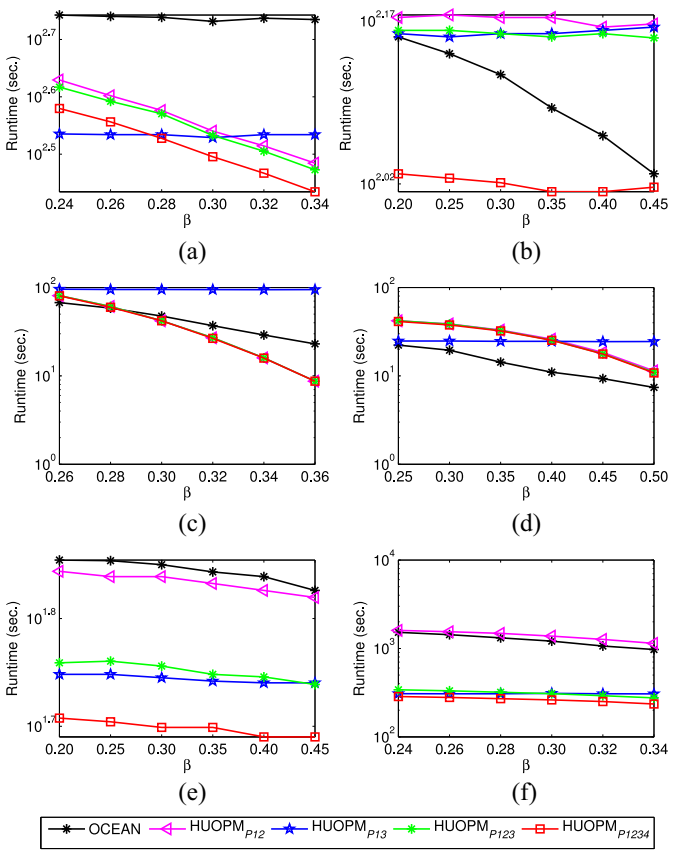


Fig. 5. Runtime under varied β with a fixed α. (a) BMSPOS2 (α:0.02%). (b) retail (α:0.014%). (c) chess (α:62%). (d) mushroom (α:18%). (e) T10I4D100K (α:0.012%). (f) T40I10D100K (α:0.8%).

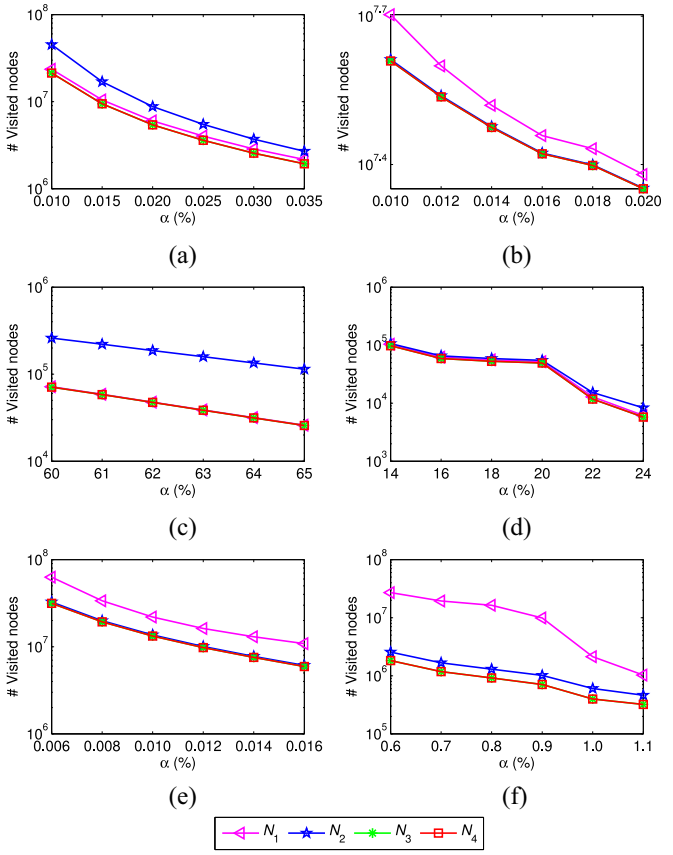


Fig. 6. Number of visited nodes under varied α with a fixed β. (a) BMSPOS2 (β:0.3). (b) retail (β:0.3). (c) chess (β:0.3). (d) mushroom (β:0.3). (e) T10I4D100K (β:0.3). (f) T40I10D100K (β:0.3).

1) The difference between HUOPM$_{P12}$ and HUOPM$_{P123}$ indicates that the Strategy 3 always reduces the search space by pruning subtrees, as it can be observed in Figs. 4 and 5.
2) By comparing HUOPM$_{P13}$ and HUOPM$_{P123}$, it can be concluded that the Strategy 2, using the upper bound of utility occupancy, provides a tradeoff between efficiency and effectiveness. For example, consider the results for BMSPOS2, shown in Fig. 5(a). When $\beta = 0.24$, HUOPM$_{P13}$ is faster than HUOPM$_{P123}$; while HUOPM$_{P13}$ consumes more runtime than HUOPM$_{P123}$ when $\beta = 0.32$ and $\beta = 0.34$. The reason for this behavior is that it needs to spend additional time to calculate the upper bounds, but sometimes unpromising itemsets can be directly pruned by other pruning strategies.
3) In general, the Strategy 4 can significantly reduce the execution time. For example, as shown in Figs. 4(b), (d), and (f) and 5(a), (b), and (e), the runtime of HUOPM$_{P1234}$ is considerably smaller than that of HUOPM$_{P123}$. Therefore, the HUOPM algorithm applies four pruning strategies to prune unpromising patterns early, which greatly speed-up the mining efficiency, compared to the baseline algorithms and the OCEAN algorithm.

Furthermore, we evaluate the statistical analysis [52] by two-way ANOVA test [53] to show whether the proposed HUOPM has a significant difference to the traditional OCEAN algorithm. Based on a two-way ANOVA analysis, in most cases, there is a significant difference of the runtime between HUOPM and OCEAN under varied α with a fixed β. Details are described below: $F = 11.551$, $P = 0.019 < 0.050$, in Fig. 4(a); $F = 156.372$, $P < 0.001$, in Fig. 4(b); $F = 3.619$, $P = 0.116$, in Fig. 4(c); $F = 8.871$, $P = 0.031 < 0.050$, in Fig. 4(d); $F = 491.368$, $P < 0.001$, in Fig. 4(e); and $F = 11.597$, $P = 0.019 < 0.050$, in Fig. 4(f). Consider Fig. 5, the results of a two-way ANOVA analysis are presented below. $F = 211.304$, $P < 0.001$, in Fig. 5(a); $F = 18.649$, $P = 0.008 < 0.010$, in Fig. 5(b); $F = 1.303$, $P = 0.305$, in Fig. 5(c); $F = 27.371$, $P = 0.003 < 0.050$, in Fig. 5(d); $F = 11705.343$, $P < 0.001$, in Fig. 5(e); and $F = 157.518$, $P < 0.001$, in Fig. 5(f). Thus, there is a significant difference of the runtime between the proposed HUOPM and the OCEAN algorithm under different parameter settings.

D. Effect of Pruning Strategies

To assess the impact of pruning strategies, we further compared the number of nodes visited in the FU-tree. The number of nodes visited by the four versions of the proposed algorithm (HUOPM$_{P12}$, HUOPM$_{P13}$, HUOPM$_{P123}$, and HUOPM$_{P1234}$) are denoted as N_1, N_2, N_3, and N_4, respectively. Results for the same parameter settings as in the previous experiments are shown in Figs. 6 and 7, respectively.

Based on Figs. 4–7, five observations are made.

1) HUOPM prunes a larger part of the search space in terms of number of nodes and is faster when α and β are

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

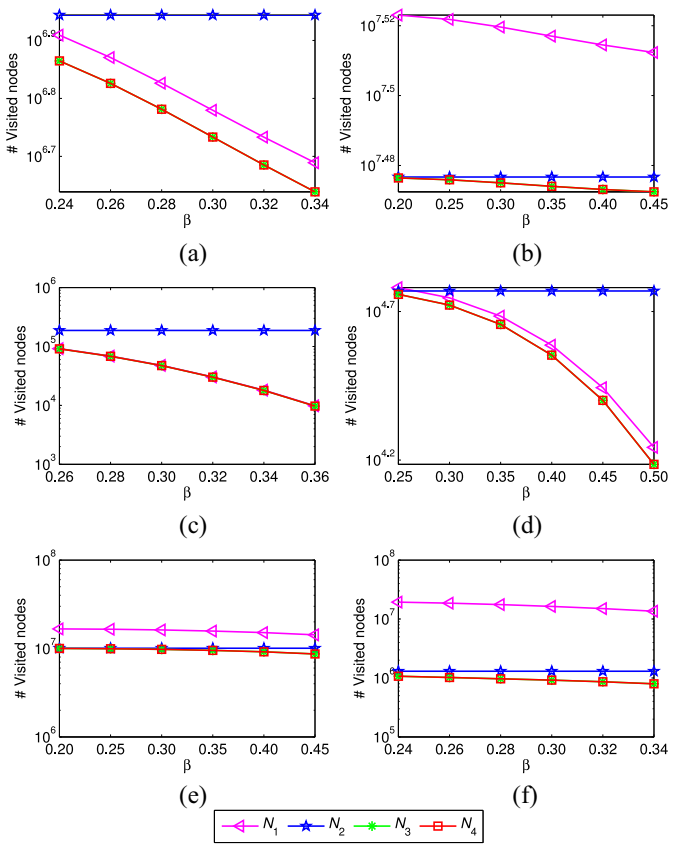


Fig. 7. Number of visited nodes under varied β with a fixed α. (a) BMSPOS2 (α:0.02%). (b) retail (α:0.014%). (c) chess (α:62%). (d) mushroom (α:18%). (e) T10I4D100K (α:0.012%). (f) T40I10D100K (α:0.8%).

increased. In general, the runtime and the number of visited nodes in the search tree of HUOPM decrease as α or β increases. Therefore, the effect of the pruning strategies on the number of visited nodes, to some extent, reflects the execution time required by the algorithms. Hence, a smaller tree is more quickly processed since less execution time is required for spanning the FU-tree and constructing UO-lists.

2) Comparing N_2 and N_3 (with and without the upper bound) indicate that the proposed upper bound is useful for pruning unpromising patterns, thus leading to a more compact search space on both sparse and dense datasets.
3) The numbers N_3 and N_4 are always the same regardless of the increase in α and β. It indicates that the Strategy 4 does not filter more unpromising patterns when compared to Strategy 3, but it has a powerful effect on reducing the runtime. We can see this trend more clearly in Figs. 4 and 5 by observing the difference between HUOPM$_{P123}$ and HUOPM$_{P1234}$. Thus, HUOPM$_{P1234}$ always consumes the least runtime among the four versions of the algorithm.
4) The higher α and β are set, the smaller the search space (number of visited nodes) is. Thus, the less runtime is required. Moreover, the less memory usage is required, but we omit the detailed memory consumption results due to the page limit.
5) The utility occupancy is a significant factor to improve the mining performance in the addressed mining task.

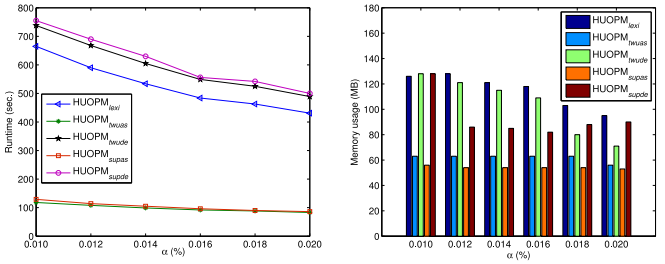


Fig. 8. Effect of processing order of items (five types of processing orders: HUOPM$_{lexi}$, HUOPM$_{twuas}$, HUOPM$_{twude}$, HUOPM$_{supas}$, and HUOPM$_{supde}$). (a) retail (β:0.3). (b) retail (β:0.3).

E. Processing Order of Items

Since the processing order of items may influence the performance of a mining algorithm, it is important to select a suitable sorting order for the proposed algorithm. To assess how different processing orders influence the performance of the proposed HUOPM algorithm, we measured the runtime and memory consumption of HUOPM with all pruning strategies but with different processing orders. Five types of processing orders are evaluated: 1) the lexicographic order (denoted as HUOPM$_{lexi}$); 2) the TWU ascending order (denoted as HUOPM$_{twuas}$); 3) the TWU descending order (denoted as HUOPM$_{twude}$); 4) the support ascending order (denoted as HUOPM$_{supas}$); and 5) the support ascending descending order (denoted as HUOPM$_{supde}$).

Fig. 8 shows the experimental results on the accidents and retail datasets, respectively. As we can see, HUOPM$_{twuas}$ has similar performance when compared to HUOPM$_{supas}$. However, HUOPM$_{twuas}$ requires more memory usage than HUOPM$_{supas}$, as shown in Fig. 8(b). Thus, the adopted support ascending order (HUOPM$_{supas}$) always leads to the best performance in terms of execution time and memory usage. Clearly, the *TWU* or support descending order provides the worst performance. It indicates that the adopted processing order with respect to support ascending order of items in the proposed HUOPM approach can greatly reduce the number of UO-lists and join operations performed for a mining task.

VI. Conclusion

In this paper, we proposed an effective and efficient HUOPM algorithm to address a new research problem of mining HUOPs with utility occupancy. The utility occupancy can lead to useful patterns that contribute a large portion of total utility for each individual transaction representing user interests or user habit. To the best of our knowledge, no prior algorithms address this problem successfully and effectively. It may provide a new research perspective for utility mining, to a certain extent. In HUOPM, an upper bound of utility occupancy for all possible extensions rooted at a processed node/itemset can be quickly derived by utilizing the UO-list. Based on the developed two *downward closure* properties and four pruning strategies, the HUOPM algorithm can directly discover the profitable HUOPs from the FU-tree using UO-list without candidate generation. Extensive experiments on several datasets show that HUOPM can efficiently find the complete set of HUOPs and significantly outperforms the

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

state-of-the-art OCEAN algorithm. Moreover, the proposed pruning strategies are powerful to prune the search space and speed-up the mining performance. Extending HUOPM to address the distributed mining problem [5], dynamic mining [40], and privacy preserving issue [54] are part of our future work.

REFERENCES

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases*, 1994, pp. 487–499.

[2] J. Han, J. Pei, Y. Yin, and R. Mao, "Mining frequent patterns without candidate generation: A frequent-pattern tree approach," *Data Min. Knowl. Disc.*, vol. 8, no. 1, pp. 53–87, 2004.

[3] L. Geng and H. J. Hamilton, "Interestingness measures for data mining: A survey," *ACM Comput. Surveys*, vol. 38, no. 3, p. 9, 2006.

[4] M.-S. Chen, J. Han, and P. S. Yu, "Data mining: An overview from a database perspective," *IEEE Trans. Knowl. Data Eng.*, vol. 8, no. 6, pp. 866–883, Dec. 1996.

[5] W. Gan, J. C. W. Lin, H. C. Chao, and J. Zhan, "Data mining in distributed environment: A survey," *Wiley Interdiscipl. Rev. Data Min. Knowl. Disc.*, vol. 7, no. 6, 2017, Art. no. e1216.

[6] K. Gouda and M. J. Zaki, "GenMax: An efficient algorithm for mining maximal frequent itemsets," *Data Min. Knowl. Disc.*, vol. 11, no. 3, pp. 223–242, 2005.

[7] E. R. Omiecinski, "Alternative interest measures for mining associations in databases," *IEEE Trans. Knowl. Data Eng.*, vol. 15, no. 1, pp. 57–69, Jan./Feb. 2003.

[8] J. Pei, J. Han, and L. V. S. Lakshmanan, "Mining frequent itemsets with convertible constraints," in *Proc. 17th Int. Conf. Data Eng.*, 2001, pp. 433–442.

[9] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Weighted frequent itemset mining over uncertain databases," *Appl. Intell.*, vol. 44, no. 1, pp. 232–250, 2016.

[10] W. Gan *et al.*, "A survey of utility-oriented pattern mining," *arXiv preprint arXiv:1805.10511*, 2018.

[11] C. F. Ahmed, S. K. Tanbeer, B. S. Jeong, and Y. K. Lee, "Efficient tree structures for high utility pattern mining in incremental databases," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 12, pp. 1708–1721, 2009.

[12] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *Proc. 21st ACM Int. Conf. Inf. Knowl. Manag.*, 2012, pp. 55–64.

[13] V. S. Tseng, C. W. Wu, B. E. Shie, and P. S. Yu, "UP-growth: An efficient algorithm for high utility itemset mining," in *Proc. 16th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2010, pp. 253–262.

[14] V. S. Tseng, B. E. Shie, C.-W. Wu, and P. S. Yu, "Efficient algorithms for mining high utility itemsets from transactional databases," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 8, pp. 1772–1786, Aug. 2013.

[15] J. C. W. Lin, L. Yang, P. Fournier-Viger, and T. P. Hong, "Mining of skyline patterns by considering both frequent and utility constraints," *Eng. Appl. Artif. Intell.*, vol. 77, pp. 229–238, Jan. 2019.

[16] R. Chan, Q. Yang, and Y. D. Shen, "Mining high utility itemsets," in *Proc. 3rd IEEE Int. Conf. Data Min.*, 2003, pp. 19–26.

[17] H. Yao, H. J. Hamilton, and C. J. Butz, "A foundational approach to mining itemset utilities from databases," in *Proc. SIAM Int. Conf. Data Min.*, 2004, pp. 482–486.

[18] L. Tang, L. Zhang, P. Luo, and M. Wang, "Incorporating occupancy into frequent pattern mining for high quality pattern recommendation," in *Proc. 21st ACM Int. Conf. Inf. Knowl. Manag.*, 2012, pp. 75–84.

[19] B. Shen, Z. Wen, Y. Zhao, D. Zhou, and W. Zheng, "OCEAN: Fast discovery of high utility occupancy itemsets," in *Proc. Pac.–Asia Conf. Knowl. Disc. Data Min.*, 2016, pp. 354–365.

[20] M. Zihayat, H. Davoudi, and A. An, "Mining significant high utility gene regulation sequential patterns," *BMC Syst. Biol.*, vol. 11, no. 6, p. 109, 2017.

[21] J. M. Luna, A. Cano, M. Pechenizkiy, and S. Ventura, "Speeding-up association rule mining with inverted index compression," *IEEE Trans. Cybern.*, vol. 46, no. 12, pp. 3059–3072, Dec. 2016.

[22] R. Srikant and R. Agrawal, "Mining quantitative association rules in large relational tables," *ACM SIGMOD Rec.*, vol. 25, no. 2, pp. 1–12, 1996.

[23] T. P. Hong, C. S. Kuo, and S. C. Chi, "Mining association rules from quantitative data," *Intell. Data Anal.*, vol. 3, no. 5, pp. 363–376, 1999.

[24] H. Verlinde, M. De Cock, and R. Boute, "Fuzzy versus quantitative association rules: A fair data-driven comparison," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 36, no. 3, pp. 679–684, 2005.

[25] H. Yao and H. J. Hamilton, "Mining itemset utilities from transaction databases," *Data Knowl. Eng.*, vol. 59, no. 3, pp. 603–626, 2006.

[26] F. Tao, F. Murtagh, and M. Farid, "Weighted association rule mining using weighted support and significance framework," in *Proc. 9th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2003, pp. 661–666.

[27] J. C. W. Lin, W. Gan, P. Fournier-Viger, and T. P. Hong, "RWFIM: Recent weighted-frequent itemsets mining," *Eng. Appl. Artif. Intell.*, vol. 45, pp. 18–32, 2015.

[28] Y. Liu, W. K. Liao, and A. Choudhary, "A two-phase algorithm for fast discovery of high utility itemsets," in *Proc. Pac.–Asia Conf. Knowl. Disc. Data Min.*, 2005, pp. 689–695.

[29] J. Liu, K. Wang, and B. C. M. Fung, "Direct discovery of high utility itemsets without candidate generation," in *Proc. IEEE 12th Int. Conf. Data Min.*, 2012, pp. 984–989.

[30] P. Fournier-Viger, C. W. Wu, S. Zida, and V. S. Tseng, "FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning," in *Proc. Int. Symp. Methodol. Intell. Syst.*, 2014, pp. 83–92.

[31] S. Krishnamoorthy, "Pruning strategies for mining high utility itemsets," *Expert Syst. Appl.*, vol. 42, no. 5, pp. 2371–2381, 2015.

[32] S. Zida, P. Fournier-Viger, J. C. W. Lin, C. W. Wu, and V. S. Tseng, "EFIM: A fast and memory efficient algorithm for high-utility itemset mining," *Knowl. Inf. Syst.*, vol. 51, no. 2, pp. 595–625, 2017.

[33] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Efficient algorithms for mining high-utility itemsets in uncertain databases," *Knowl. Based Syst.*, vol. 96, pp. 171–187, Mar. 2016.

[34] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Efficiently mining uncertain high-utility itemsets," *Soft Comput.*, vol. 21, no. 11, pp. 2801–2820, 2017.

[35] J. C. W. Lin, W. Gan, T. P. Hong, and V. S. Tseng, "Efficient algorithms for mining up-to-date high-utility patterns," *Adv. Eng. Informat.*, vol. 29, no. 3, pp. 648–661, 2015.

[36] J. C. W. Lin, J. Zhang, P. Fournier-Viger, T. P. Hong, and J. Zhang, "A two-phase approach to mine short-period high-utility itemsets in transactional databases," *Adv. Eng. Informat.*, vol. 33, pp. 29–43, Aug. 2017.

[37] J. C. W. Lin, P. Fournier-Viger, and W. Gan, "FHN: An efficient algorithm for mining high-utility itemsets with negative unit profits," *Knowl. Based Syst.*, vol. 111, pp. 283–298, Nov. 2016.

[38] J. C. W. Lin, W. Gan, T. P. Hong, and B. Zhang, "An incremental high-utility mining algorithm with transaction insertion," *Sci. World J.*, vol. 2015, p. 15, Sep. 2015.

[39] J. C. W. Lin, W. Gan, and T. P. Hong, "A fast maintenance algorithm of the discovered high-utility itemsets with transaction deletion," *Intell. Data Anal.*, vol. 20, no. 4, pp. 891–913, 2016.

[40] W. Gan *et al.*, "A survey of incremental high-utility itemset mining," *Wiley Interdiscipl. Rev. Data Min. Knowl. Disc.*, vol. 8, no. 2, 2018, Art. no. e1242.

[41] U. Yun, G. Lee, and E. Yoon, "Efficient high utility pattern mining for establishing manufacturing plans with sliding window control," *IEEE Trans. Ind. Electron.*, vol. 64, no. 9, pp. 7239–7249, Sep. 2017.

[42] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Mining high-utility itemsets with various discount strategies," in *Proc. IEEE Int. Conf. Data Sci. Adv. Anal.*, 2015, pp. 1–10.

[43] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Fast algorithms for mining high-utility itemsets with various discount strategies," *Adv. Eng. Informat.*, vol. 30, no. 2, pp. 109–126, 2016.

[44] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and H. Fujita, "Extracting non-redundant correlated purchase behaviors by utility measure," *Knowl. Based Syst.*, vol. 143, pp. 30–41, Mar. 2018.

[45] V. S. Tseng, C. W. Wu, P. Fournier-Viger, and P. S. Yu, "Efficient algorithms for mining top-*K* high utility itemsets," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 1, pp. 54–67, 2016.

[46] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and H. C. Chao, "FDHUP: Fast algorithm for mining discriminative high utility patterns," *Knowl. Inf. Syst.*, vol. 51, no. 3, pp. 873–909, 2017.

[47] Y. C. Lin, C. W. Wu, and V. S. Tseng, "Mining high utility itemsets in big data," in *Proc. Pac.–Asia Conf. Knowl. Disc. Data Min.*, 2015, pp. 649–661.

[48] N. Pasquier, Y. Bastide, R. Taouil, and L. Lakhal, "Pruning closed itemset lattices for association rules," in *Proc. Int. Conf. Adv. Databases*, 1998, pp. 177–196.

[49] R. Rymon, "Search through systematic set enumeration," in *Proc. 3rd Int. Conf. Principles Knowl. Represent. Reasoning*, 1992, pp. 539–550.

[50] (2012). *Frequent Itemset Mining Dataset Repository*. [Online]. Available: http://fimi.ua.ac.be/data/

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[51] (1994). *Quest Synthetic Data Generator*. [Online]. Available: http://www.Almaden.ibm.com/cs/quest/syndata.html
[52] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
[53] T. H. Wonnacott and R. J. Wonnacott, *Introductory Statistics*. New York, NY, USA: Wiley, 1990.
[54] W. Gan, J. C. W. Lin, H. C. Chao, S. L. Wang, and P. S. Yu, "Privacy preserving utility mining: A survey," *arXiv preprint arXiv:1811.07389*, 2018.



Wensheng Gan received the B.S. degree in computer science from South China Normal University, Guangzhou, China, in 2013 and the M.S. degree in computer science from the Harbin Institute of Technology (Shenzhen), Shenzhen, China, in 2015. He is currently pursuing the joint Ph.D. degree with the Department of Computer Science, University of Illinois at Chicago, Chicago, IL, USA, and with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), Shenzhen, China.

He has published over 50 research papers in peer-reviewed journals and international conferences. His current research interests include data mining, utility mining, and big data technologies.



Jerry Chun-Wei Lin (M'08) received the Ph.D. degree in computer science and information engineering from National Cheng Kung University, Tainan, Taiwan, in 2010.

He is an Associate Professor with the Department of Computing, Mathematics and Physics, Western Norway University of Applied Sciences, Bergen, Noway. He is the Co-Leader of the popular SPMF open-source data mining library and the Founder of the PPSF open-source privacy-preserving and security project. He has published over 250 research papers in peer-reviewed international conferences and journals. His current research interests include data mining, privacy-preserving and security, big data analytics, and machine learning.

Dr. Lin was a recipient of the Chinese Most Cited Researchers Award in 2018. He is the Editor-in-Chief of the *Data Mining and Pattern Recognition* journal.



Philippe Fournier-Viger received the Ph.D. degree in computer science from the University of Quebec, Montreal, QC, Canada, in 2010.

He is a Full Professor and a Youth 1000 Scholar with the Harbin Institute of Technology (Shenzhen), Shenzhen, China. He has published over 200 research papers in peer-reviewed international conferences and journals. He has been the Founder of the popular SPMF open-source data-mining library, which has been cited in over 700 research papers since 2010. His current research interests include data mining, pattern mining, sequence analysis and prediction, e-learning, and social network mining.

Dr. Fournier-Viger is the Editor-in-Chief of the *Data Mining and Pattern Recognition* journal.



Han-Chieh Chao (SM'04) received the M.S. and Ph.D. degrees in electrical engineering from Purdue University, West Lafayette, IN, USA, in 1989 and 1993, respectively.

Since 2016, he has been the President of the National Dong Hwa University, Hualien, Taiwan. He has published nearly 500 peer-reviewed professional research papers. His current research interests include high-speed networks, wireless networks, IPv6-based networks, and artificial intelligence.

Dr. Chao is the Editor-in-Chief of IET Networks and *Journal of Internet Technology*. He is the Founding Editor-in-Chief of the *International Journal of Internet Protocol Technology* and the *International Journal of Ad Hoc and Ubiquitous Computing*. He has served as the Guest Editor for ACM MONET, the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, *IEEE Communications Magazine*, *IEEE Systems Journal*, *Computer Communications*, IEEE PROCEEDINGS COMMUNICATIONS, *Wireless Personal Communications*, and *Wireless Communications & Mobile Computing*. He is a fellow of IET.



Philip S. Yu (F'93) received the B.S. degree in electrical engineering from National Taiwan University, Taipei, Taiwan, the M.S. and Ph.D. degrees in electrical engineering from Stanford University, Stanford, CA, USA, and the M.B.A. degree from New York University, New York, NY, USA.

He was with IBM, Armonk, NY, USA, where he was a Manager of the Software Tools and Techniques Department with the Thomas J. Watson Research Center. He is a Distinguished Professor of computer science with the University of Illinois at Chicago, Chicago, IL, USA, where he also holds the Wexler Chair in information technology. He has published over 1200 papers in peer-reviewed journals, such as the IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, *ACM Transactions on Knowledge Discovery from Data*, VLDBJ, and the *ACM Transactions on Intelligent Systems and Technology* and conferences, such as KDD, ICDE, WWW, AAAI, SIGIR, ICML, and CIKM. He holds or has applied for over 300 U.S. patents. His current research interests include data mining, data streams, databases, and privacy.

Dr. Yu was a recipient of the ACM SIGKDD 2016 Innovation Award for his influential research and scientific contributions on mining, fusion, and anonymization of Big Data and the IEEE Computer Society 2013 Technical Achievement Award. He is the Editor-in-Chief of *ACM Transactions on Knowledge Discovery from Data*. He is a fellow of ACM.